

• Producteur-Consommateur (Bounded Buffer)

```

1  #include <semaphore.h>
2  #include <unistd.h>
3  #include <pthread.h>
4  #include <stdio.h>
5
6  #define taille 3 // Taille maximale du tampon circulaire
7  sem_t plein, vide, mutex; // Déclaration des sémaphores de synchronisation
8  int tampon[taille]; // Tampon partagé (Bounded buffer)
9  pthread_t cons, prod; // Déclaration des threads
10 void* consommateur(void *);
11 void* producteur(void *);
12
13 int main(int argc, char *argv[]){
14     sem_init(&plein,0,0); // Init plein: 0 élément au départ
15     sem_init(&vide,0,taille); // Init vide: 3 places libres au départ
16     sem_init(&mutex,0,1); // Init mutex: 1 (accès exclusif libre)
17     pthread_create(&cons, NULL, consommateur, NULL); // Création du thread consommateur
18     pthread_create(&prod, NULL, producteur, NULL); // Création du thread producteur
19
20     pthread_join(prod,NULL); // Le main attend la fin du producteur
21     pthread_join(cons,NULL); // Le main attend la fin du consommateur
22
23     printf("Fin des threads\n");
24     return 0;
25 }
26 void* consommateur(void *){
27     int ic=0, nbcons=0, objet; // ic = index, nbcons = compteur d'arrêts
28     do{
29         sem_wait(&plein); // Attendre qu'il y ait un item disponible (P)
30         sem_wait(&mutex); // Verrouiller l'accès exclusif au tampon (P)
31         objet=tampon[ic]; // [SECTION CRITIQUE] Lecture de la donnée
32         sem_post(&mutex); // Déverrouiller l'accès au tampon (V)
33         sem_post(&vide); // Signaler qu'une place s'est libérée (V)
34         ic = (ic + 1) % taille; // Avancer l'index circulairement (0, 1, 2, 0...)
35         nbcons++; // Incréments la condition de sortie
36         sleep(2); // Simuler le temps de traitement/consumation
37     }while(nbcons<=5); // Boucle pour consommer 6 éléments (0 à 5)
38     return NULL;
39 }
40 void* producteur(void *){
41     int ip=0, nbprod=0, objet=0; // ip = index, objet = donnée à insérer
42     do{
43         sem_wait(&vide); // Attendre qu'il y ait une place libre (P)
44         sem_wait(&mutex); // Verrouiller l'accès exclusif au tampon (P)
45         tampon[ip] = objet; // [SECTION CRITIQUE] Écriture de la donnée
46         sem_post(&mutex); // Déverrouiller l'accès au tampon (V)
47         sem_post(&plein); // Signaler qu'un nouvel item est dispo (V)
48         objet++; // Générer la donnée suivante
49         nbprod++; // Incréments la condition de sortie
50         ip = (ip + 1) % taille; // Avancer l'index circulairement (0, 1, 2, 0...)
51     }while(nbprod <= 5); // Boucle pour produire 6 éléments (0 à 5)
52     return NULL;
53 }

```

• Le cycle de vie d'un processus (5 states):

- New -> Ready -> In execution -> Waiting (or terminated)
- Piège QCM: Une opération E/S met un processus en attente. La fin de son temps alloué (quantum) le remet à prêt.

• context switch: C'est le fait de save le PCG d'un processus pour sauvegarder son état et en charger un autre

- point clé: C'est du gaspillage pur (overhead) pour le système; aucun travail n'est fait pendant ce temps

• Les 3 ordonnanceurs:

- Long terme: Gère le nombre de processus en mémoire (degré de multiprogrammation). Il est très lent
- Court terme: Donne le CPU à un processus, il tourne en milliseconde (très rapide)
- Moyen terme: fait du "swapping", (sort des processus de la mémoire vers le disk (swap) pour faire de la place).

• Les 3 règles d'une section critique valide:

1. Exclusion mutuelle
2. Progres (Pas de report indéfini)
3. Attente limitée (pas de famine)

Mutex vs spinlock: Le mutex endort le processus s'il est bloqué (provoque un context switch coûteux). Le spinlock fait une "attente-active" (tourne en boucle). Il gaspille du CPU, mais il est génial si l'attente est ultra-courte car il évite de payer le prix d'un context switch.

• LECTEURS - ECRIVAINS (Priorite aux lecteurs)

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #define N 3 // Nombre de threads lecteurs et rédacteurs
6  #define ACCES 4 // Nombre d'accès (itérations) par thread
7  int Nbl = 0; // read_count: Nombre de lecteurs actifs
8  sem_t Redact; // rw_mutex: Accès exclusif à la ressource partagée
9  sem_t mutex; // Protège l'accès à la variable partagée Nbl
10 void * lecteur(void *);
11 void * redacteur(void *);
12 int main(int argc, char *argv[]){
13     int i;
14     int numred[N] = {0,1,2}; // Identifiants à passer aux threads rédacteurs
15     int numlec[N] = {0,1,2}; // Identifiants à passer aux threads lecteurs
16     pthread_t red[N]; // Tableau pour stocker les IDs des rédacteurs
17     pthread_t lec[N]; // Tableau pour stocker les IDs des lecteurs
18     sem_init(&mutex, 0, 1); // Init mutex: 1 (compteur Nbl libre d'accès)
19     sem_init(&Redact, 0, 1); // Init rw_mutex: 1 (ressource libre d'accès)
20     for(i=0; i<N; i++){
21         pthread_create(&lec[i], NULL, lecteur, &(numlec[i])); // Création du thread lecteur
22         pthread_create(&red[i], NULL, redacteur, &(numred[i])); // Création du thread rédacteur
23     }
24     for(i=0; i<N; i++){
25         pthread_join(red[i], NULL); // Le main attend la fin du rédacteur i
26         pthread_join(lec[i], NULL); // Le main attend la fin du lecteur i
27     }
28     printf("Fin des thread\n");
29     return 0;
30 }
31 void * lecteur(void * num){
32     int i=(int *) num; // Cast et récupération de l'ID du lecteur
33     int x=ACCES; // Initialise le compteur de boucle (4 itérations)
34     do{
35         sem_wait(&mutex); // Verrouille l'accès au compteur Nbl (P)
36         if(!Nbl) // Si c'est le 1er lecteur (Nbl == 0)...
37             sem_wait(&Redact); // ...il barre la porte d'accès aux rédacteurs
38         Nbl++; // Incrémente le nombre de lecteurs actifs
39         sem_post(&mutex); // Déverrouille l'accès au compteur Nbl (V)
40         sleep(1); // [SECTION CRITIQUE] Lecture concurrente permise
41         sem_wait(&mutex); // Verrouille l'accès au compteur Nbl (P)
42         Nbl--; // Décrémente le nombre de lecteurs actifs
43         if(!Nbl) // Si c'est le dernier lecteur (Nbl == 0)...
44             sem_post(&Redact); // ...il débloque la porte pour les rédacteurs
45         sem_post(&mutex); // Déverrouille l'accès au compteur Nbl (V)
46         sleep(1); // Temps d'attente hors section critique
47     }while(--x); // Décrémente x et continue tant que > 0
48 }
49 void * redacteur(void * num){
50     int i = *(int *)num; // Cast et récupération de l'ID du rédacteur
51     int x=ACCES; // Initialise le compteur de boucle (4 itérations)
52     do{
53         sem_wait(&Redact); // Demande l'accès EXCLUSIF total (bloque tout)
54         sleep(1); // [SECTION CRITIQUE] Écriture exclusive en cours
55         sem_post(&Redact); // Libère l'accès exclusif total
56     }while(--x); // Décrémente x et continue tant que > 0
57 }

```

- **fork**: Si un thread appelle fork(), est-ce que ça duplique tous les thread? Non juste l'appelant en UNIX

• Pthreads

- pthread_create(&tid, attr, fonction, arg) : Lance un thread. Note bien le 4ème argument arg : on ne peut passer qu'un seul pointeur void*. Si tu veux passer plusieurs arguments, tu dois créer une struct et passer son pointeur.

pthread_join(tid, &status) : C'est l'équivalent du wait() pour les processus. Le thread appelant bloque jusqu'à ce que le thread tid ait fini.

pthread_exit(valeur) : Arrête le thread proprement.

• Dining philosophers (version moniteur complet)

```

1  monitor DiningPhilosophers{
2      enum { THINKING, HUNGRY, EATING } state[5];
3      condition self[5];
4      void pickup(int i) {
5          state[i] = HUNGRY;
6          test(i);
7          if (state[i] != EATING) {
8              self[i].wait();
9          }
10     }
11     void putdown(int i) {
12         state[i] = THINKING;
13         // test left and right neighbors
14         test((i+4)%5);
15         test((i+1)%5);
16     }
17     void test(int i) {
18         if ((state[(i+4)%5] != EATING) &&
19             (state[i] == HUNGRY) &&
20             (state[(i+1)%5] != EATING) ) {
21             state[i] = EATING;
22             self[i].signal();
23         }
24     }
25     initialization_code() {
26         for (int i=0; i<5; i++)
27             state[i] = THINKING;
28     }
29 }

```

• Implementation d'un moniteur via semaphores

```

1  // VARIABLES GLOBALES (Initialisation)
2  semaphore mutex; // initialisé à 1 - contrôle l'accès au moniteur
3  semaphore next; // initialisé à 0 - redonne le contrôle après un signal
4  int next_count = 0; // nombre de processus suspendus par les conditions
5  wait(mutex);
6  body of F; // REMPLACEMENT DE CHAQUE FONCTION "F" DU MONITEUR
7  if (next_count > 0)
8      signal(next); else signal(mutex);
9  // POUR CHAQUE VARIABLE DE CONDITION "x"
10 semaphore x_sem; // initialisé à 0
11 int x_count = 0; // nombre de processus bloqués sur x
12 // IMPLÉMENTATION DE L'OPÉRATION "x.wait()"
13 x_count++;
14 if (next_count > 0)
15     signal(next); else signal(mutex);
16 wait(x_sem);
17 x_count--;
18 // IMPLÉMENTATION DE L'OPÉRATION "x.signal()"
19 if (x_count > 0) {
20     next_count++;
21     signal(x_sem);
22     wait(next);
23     next_count--;
24 } // else fait rien

```

• Les 3 modeles de multithreading:

- Plusieurs à un (many to one): Tous les threads utilisateurs sont mappés sur 1 seul thread noyau. Le probleme fatal est que si un seul thread fait un appel bloquant, **tout le processus bloque (les autres threads s'arretent aussi)** car le noyau ne voit qu'un seul thread. ZZero parallelisme
- One to one: 1 thread user = 1 thread noyau
 - C'est le modele linux/Windows actuel. L'avantage: Du vrai parallélisme, si un thread bloque, les autres continuent. Desavantage: Créer un thread user coute plus cher car il faut aussi crer un thread noyau
- Many-to-Many: On multiplexe M threads user sur N threads noyau ou M>=N. C'est le plus flexible mais le plus complexe a gerer

• La loi d'Amdahl (calcul/QCM)

$\text{speedup} \leq \frac{1}{S + \frac{1-S}{N}}$ C'est la formule qui dit "Ca ne sert a rien d'ajouter 1000 coeurs si ton code est sequentiel".

- S: La portion de ton code qui obligatoirement sequentielle (ne peut pas etre parallelise)
- N: le nombre de coeurs
- Conclusion pour l'examen: Si S est grand, ajouter des coeur (N) ne change presque rien, la vitesse maximale theorique est 1/S.

```

1 #define BUFFER_SIZE 1024 // Max buffer to exchange string
2 int main(){
3     const char *name = "My_Shared_Memory_Space";
4     int sharedMemFileDescriptor;
5     void *ptr; //opaque ptr toward the object in shared memory
6     // Creating the actual space
7     sharedMemFileDescriptor = shm_open(name, O_RDONLY, 0666);
8     // Setting the size of the file that is referenced by the FileDescriptor (a number)
9     // In this case, we set it to a size of 1024 bytes, so a 1 kilo byte (1 KB)
10    ftruncate(sharedMemFileDescriptor, BUFFER_SIZE);
11    ptr = mmap(0, BUFFER_SIZE, PROT_READ, MAP_SHARED, sharedMemFileDescriptor, 0);
12    printf("%s", (char *)ptr);
13    shm_unlink(name);
14    return 0;
15 }
16 //sender
17 int main(){
18     const char *name = "My_Shared_Memory_Space";
19     const char *firstMessage = "This is a sequence ";
20     const char *secondMessage = "of bytes.";
21     int sharedMemFileDescriptor;
22     void *ptr; //opaque ptr toward the object in shared memory
23
24
25     // Creating the actual space
26     sharedMemFileDescriptor = shm_open(name, O_CREAT | O_RDWR, 0666);
27
28     // Setting the size of the file that is referenced by the FileDescriptor (a number)
29     // In this case, we set it to a size of 1024 bytes, so a 1 kilo byte (1 KB)
30    ftruncate(sharedMemFileDescriptor, BUFFER_SIZE);
31
32    ptr = mmap(0, BUFFER_SIZE, PROT_WRITE, MAP_SHARED, sharedMemFileDescriptor, 0);
33
34    sprintf(ptr, "%s", firstMessage);
35    ptr += strlen(firstMessage);
36    sprintf(ptr, "%s", secondMessage);
37 }
38

```

- **Famine vs Interblocage:**

- Famine: Un processus n'a jamais son tour, mais le reste du système continue de fonctionner (à cause de l'ordonnanceur)
- Interblocage: Une attente circulaire ou TOUT est bloqué de façon permanente. (2 processus waiting for each others for example)

- **Moniteur vs Semaphore:**

- Le moniteur est une "bulle" de haut niveau qui encapsule les données et où un seul processus entre à la fois.
 - Différence vitale: Un signal() sur un semaphore incrémente un compteur (il est mémorisé). Un signal() sur une variable de condition dans un moniteur, s'il n'y a personne en attente, est perdu (il ne fait rien).

- **IPC: Mémoire partagée vs message passing**

- SHM: très rapide, pas d'overhead du noyau après création, mais c'est au programmeur de coder la synchronisation manuellement (risky).
 - Message passing (Ex: Pipes): Plus lent (passe par le noyau), mais la synchronisation est souvent gérée automatiquement par le système

- **Les tubes nommés (FIFO):** Sont de vrais fichiers sur le disque dur. Pas besoin de relation-parent enfant pour être utilisé comme pour les tubes ordinaires. N'importe quel processus qui connaît le nom du fichier peut s'y brancher.

